CS 422 HW2 Report

Repository: CS422-Networks-Group/HW2. Code links point to the `main` branch. Part 1 and 2 results come from commit `54b78a9` (same measurement code, CUBIC only); Part 3 results are the current files in `results/`.

Setup and reproduction

- Entry point: `src/main.py`. All command-line options are documented in the README.
- Server list: `data/listed_iperf3_servers.csv`, taken from iperf3serverlist.net.
- Dependencies: Python 3, `pandas==3.0.6`, `matplotlib==3.11.2` (requirements.txt).
- Host kernel: `6.6.87.2-microsoft-standard-WSL2` (Windows, WSL2). The script records this in `run_metadata.json`.
- Raw output: `results/` (`tcp_stats.csv`, `goodput.csv`, `summary.csv`, `run_metadata.json`, and the PDF plots).

The Part 1 and 2 results came from this command (session `20261018T201911`):

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python src/main.py data/listed_iperf3_servers.csv --n 10 --time 60 --interval 1.0 --out results
```

Docker

The `Dockerfile` is based on `ubuntu:24.04` and runs `src/main.py`, so running the container with no arguments runs the whole pipeline (CUBIC, Reno, and BBR tests, CSVs, summary tables, and plots). Since the container uses the host kernel, the host has to load BBR and allow unprivileged sockets to pick each algorithm first:

```
sudo modprobe tcp_bbr
sudo sysctl -w net.ipv4.tcp_allowed_congestion_control="reno cubic bbr"
```

Then, from the repo root:

```
docker build -t hw2 .
docker run --rm --network host -v "$PWD/results:/hw2/results" hw2
```

Arguments after the image name are passed to `main.py`.

Part 1: iPerf throughput application

(a) iperf3-compatible client

The client talks to the iperf3 server over plain sockets and never calls the `iperf3` binary. One test is handled by `run_test()`:

1. **Control connection** (L125–L138): connect over TCP and send a random 36-character cookie followed by a null byte.
2. **Parameter exchange** (L140–L147): wait for `PARAM_EXCHANGE`, then send a 4-byte length and the JSON test parameters.
3. **Data connection** (L149–L158): on `CREATE_STREAMS`, open a second TCP connection and send the same cookie.
4. **Transmission** (L160–L175): after `TEST_START` and `TEST_RUNNING`, send 128 KiB blocks until `--time` seconds have passed.
5. **Termination** (L177–L206): send `TEST_END`, exchange JSON results, read `DISPLAY_RESULTS`, send `IPERF_DONE`, and close both sockets.

Every failure raises an exception that `main()` logs before moving on to the next server. A busy or overloaded server replies with `ACCESS_DENIED` instead of `PARAM_EXCHANGE`, and a server that rejects the parameters replies with something other than `CREATE_STREAMS`; both fail the state check. Both sockets have a 10 s timeout, so a server that never answers raises `socket.timeout` (66.35.20.123:30001 failed this way in our run). If the server closes the connection early, the next read comes back empty or raises `ConnectionResetError`.

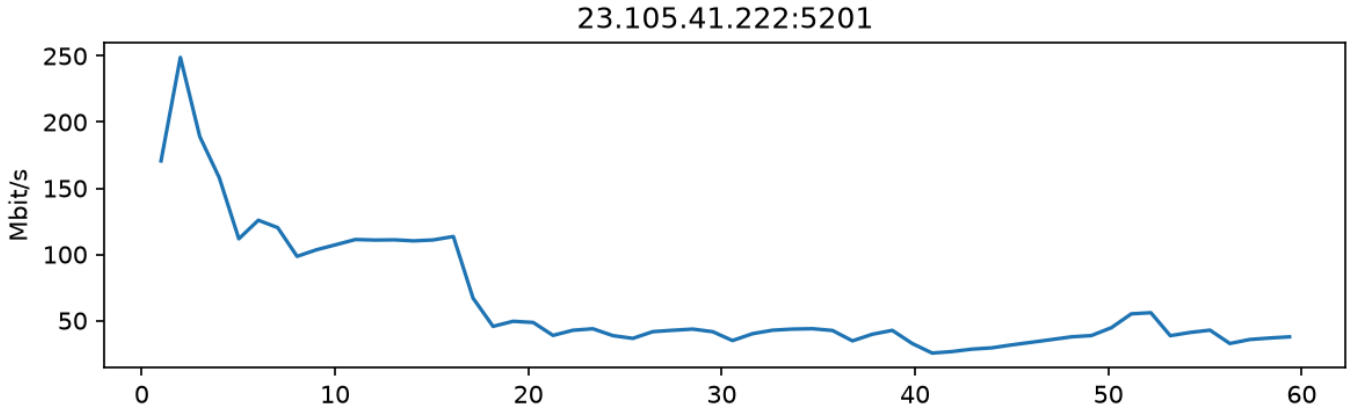
(b) Destination selection

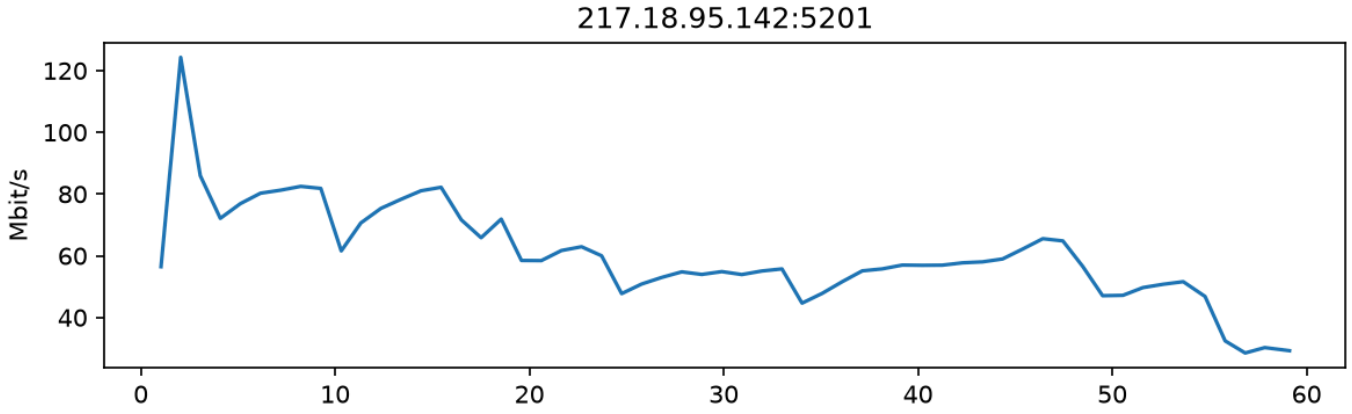
`main()` shuffles the server list and tries servers in that order until `--n` tests succeed or `--max-attempts` (default 30) servers have been tried. Each attempt is logged in `run_metadata.json` with its address, port, and outcome, and a shortfall prints `shortfall: only k of n servers succeeded`. Our run asked for 10 servers and got 10 after 11 attempts, so there was no shortfall.

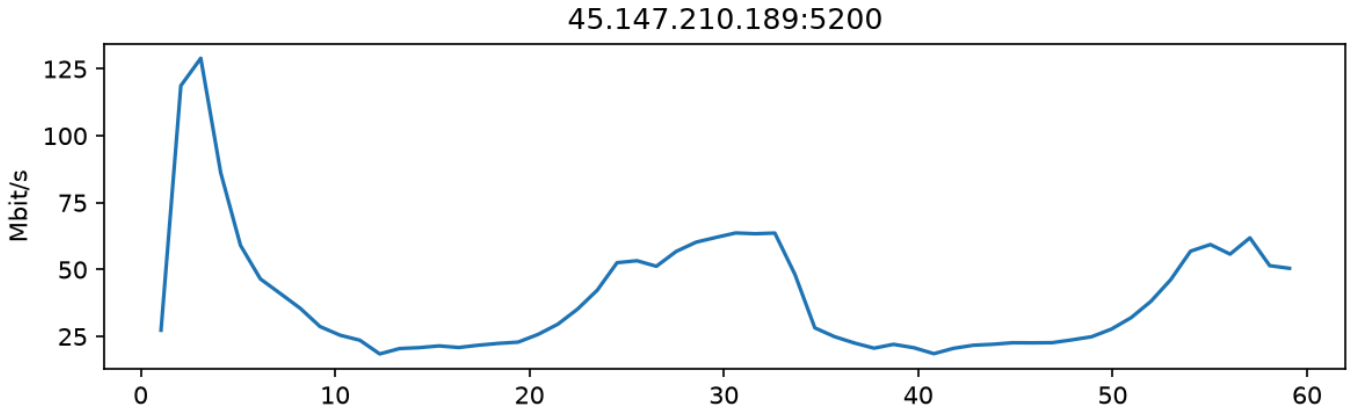
(c) Goodput

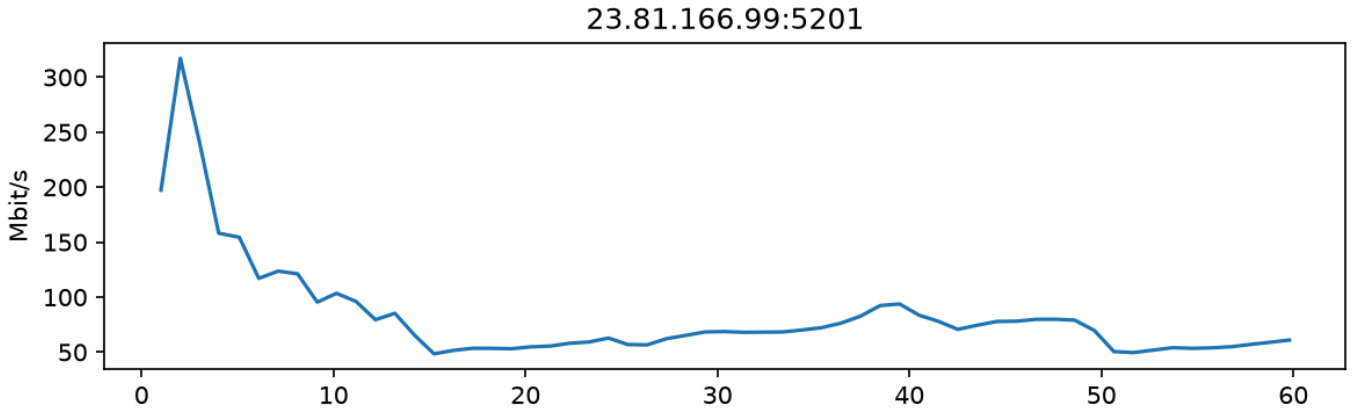
Goodput is computed in `tcp_sample()` from the kernel's cumulative `tcpi_bytes_acked` on the data socket, divided by the actual time between reads:

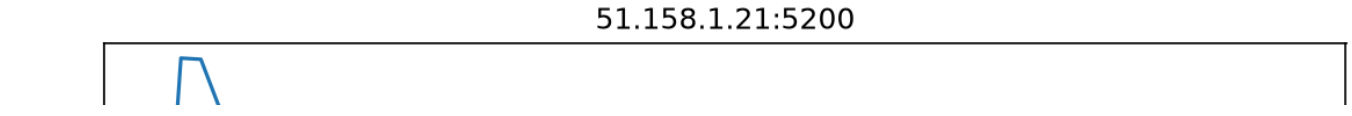
$$\text{goodput}_k = \frac{(\text{bytes_acked}_k - \text{bytes_acked}_{k-1}) \cdot 8}{t_k - t_{k-1}}$$

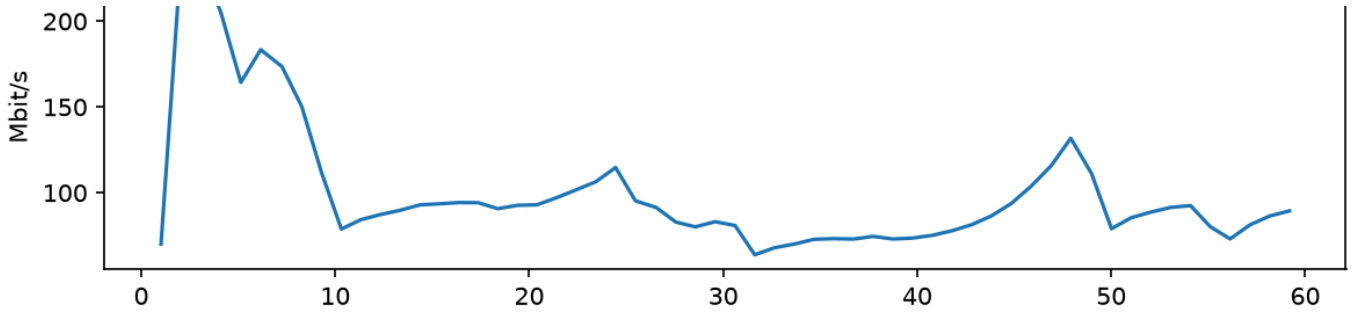

23.105.41.222:5201
Mbit/s
250
200
150
100
50
0
10
20
30
40
50
60


217.18.95.142:5201
Mbit/s
120
100
80
60
40
0
10
20
30
40
50
60

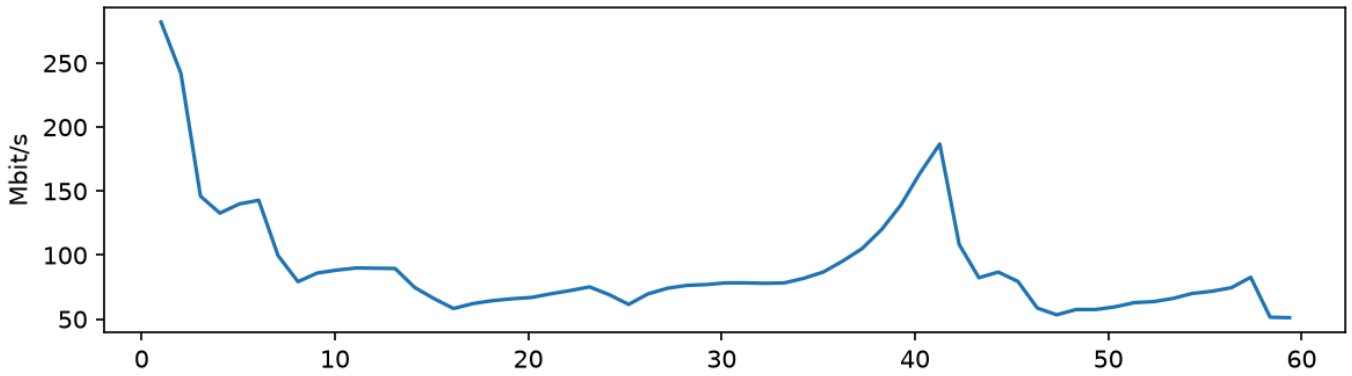

45.147.210.189:5200
Mbit/s
125
100
75
50
25
0
10
20
30
40
50
60


23.81.166.99:5201
Mbit/s
300
250
200
150
100
50
0
10
20
30
40
50
60

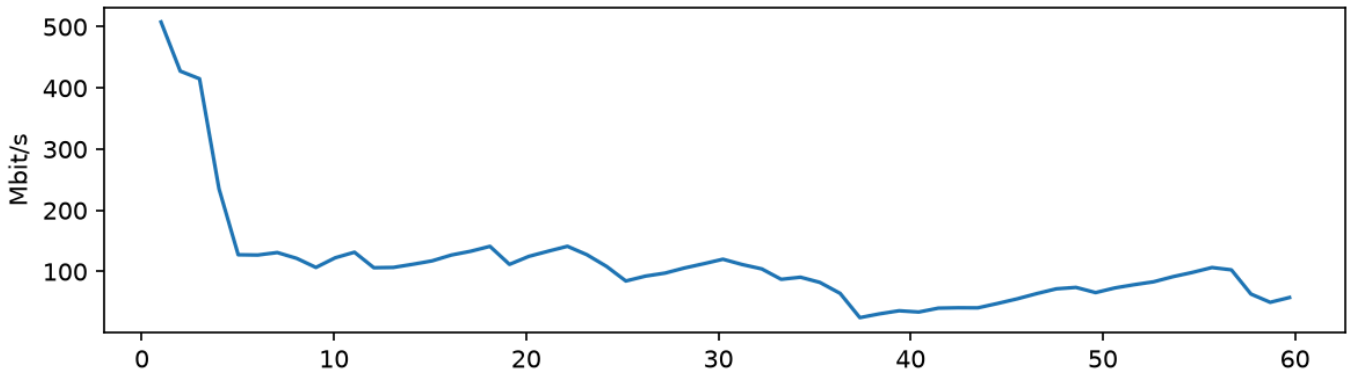

51.158.1.21:5200



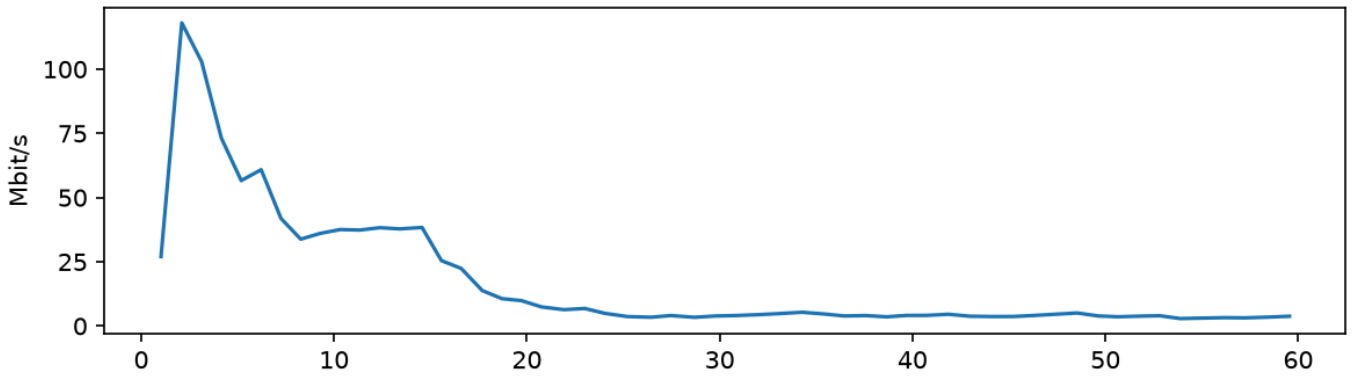
23.81.207.31:5201



157.250.201.61:5203

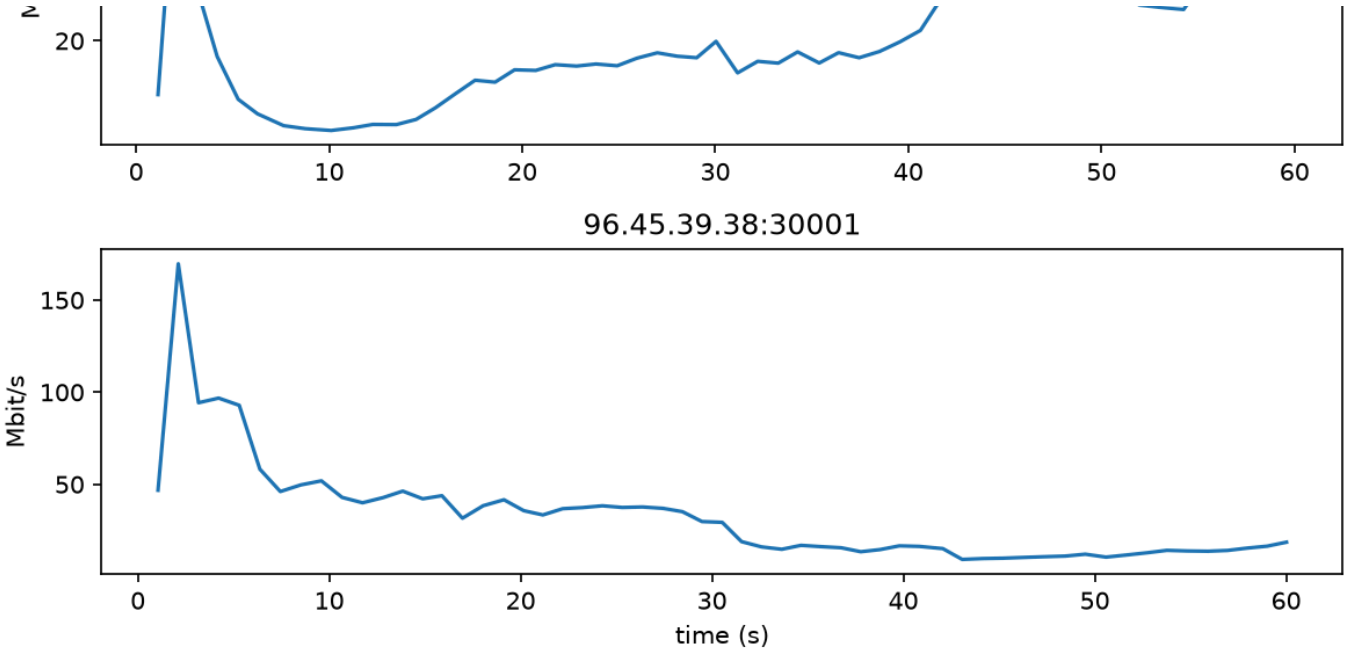


156.146.40.65:5201



185.246.208.67:5201





Goodput over 60 s for each destination (CUBIC, 1 s samples), `results/goodput.pdf`*. Generated by* `plot_goodput()`*, called once per algorithm from* `main()`*.*

Summary in Mbit/s (`results/summary.csv` at `54b78a9`; the Part 3 table is below):

Destination	Algorithm	Min	Median	Mean	p95
157.250.201.61:5203	cubic	24.7	104.3	112.4	253.3
51.158.1.21:5200	cubic	63.8	89.6	100.4	186.7
23.81.207.31:5201	cubic	51.1	78.0	90.5	166.7
23.81.166.99:5201	cubic	48.4	68.7	82.9	162.0
23.105.41.222:5201	cubic	25.4	42.7	65.3	159.9
217.18.95.142:5201	cubic	28.5	57.0	60.4	82.2
45.147.210.189:5200	cubic	18.5	29.1	39.9	67.1
96.45.39.38:30001	cubic	9.2	29.4	32.5	93.1
185.246.208.67:5201	cubic	3.1	17.1	19.9	35.6
156.146.40.65:5201	cubic	2.9	4.6	17.7	64.5

Part 2: TCP stats tracing

All Part 2 runs used `cubic`, the kernel default. We read the algorithm back from the data socket with `getsockopt(TCP_CONGESTION)` and store it in the `algorithm` column of every row.

(a) Extracting TCP socket statistics

The client reads `TCP_INFO` from the data socket once before sending, then after each `sendall()` once at least `--interval` (1 s) has passed (L164–L175). Goodput and the TCP stats come from the same read, so they share a timestamp.

Decoding format. `tcp_info()` unpacks the buffer as Linux `struct tcp_info` (from `include/uapi/linux/tcp.h`) up to `tcpi_snd_wnd`, which is 232 bytes on kernel 5.4 and newer. The format string and field names are at L14–L29:

```
TCP_INFO_FMT = "=8B24I4Q6IQ3Q2I2Q2I2I"   # 232 bytes, kernel 6.6.87.2-microsoft-standard-WSL2
```

Format	Fields
`8B`	state, ca_state, retransmits, probes, backoff, options, wscale, flags
`24I`	rto, ato, snd_mss, rcv_mss, unacked, sacked, lost, retrans, fackets, last_data_sent, last_ack_sent, last_data_recv, last_ack_recv, pmtu, rcv_ssthresh, rtt, rttvar, snd_ssthresh, snd_cwnd, advmss, reordering, rcv_rtt, rcv_space, total_retrans
`4Q`	pacing_rate, max_pacing_rate, bytes_acked, bytes_received
`6I`	segs_out, segs_in, notsent_bytes, min_rtt, data_segs_in, data_segs_out
`Q 3Q`	delivery_rate, busy_time, rwnd_limited, sndbuf_limited
`2I 2Q`	delivered, delivered_ce, bytes_sent, bytes_retrans
`2I 2I`	dsack_dups, reord_seen, rcv_ooopack, snd_wnd

Interval measurements. `tcp_sample()` takes the difference of each cumulative counter between consecutive reads and divides by the measured gap `dt`. A 128 KiB `sendall()` can block, so the real gap was 1.000–1.354 s rather than exactly 1 s. Goodput is Δ`bytes_acked` × 8 / dt. Fields like cwnd, srtt, and rttvar are just the value at the time of the read.

Loss proxy. TCP can't observe loss directly, so we use the number of retransmitted segments per interval (Δ`total_retrans`). This overcounts when retransmissions are spurious, e.g. after reordering. We also log `lost_segs`, `rto_backoffs` (nonzero during an unrecovered timeout), and `ca_state` as supporting signals.

Storage. `results/tcp_stats.csv` has one row per sample with `run_id`, `destination`, `algorithm`, a timestamp, goodput, and the decoded stats. Units are in the column name suffix: `_s`, `_ms`, `_bps`, `_bytes`, `_segs`.

(b) Visualization

`pick_representative()` picks the destination with the median mean goodput, which was 217.18.95.142:5201 (60.4 Mbit/s mean, 93.0 ms min RTT). `plot_tcp_stats()` generates `tcp_stats_timeseries.pdf` (time series, L276–L294) and `tcp_stats_scatter.pdf` (scatter, L296–L306) at the end of each run, called from `main()`.

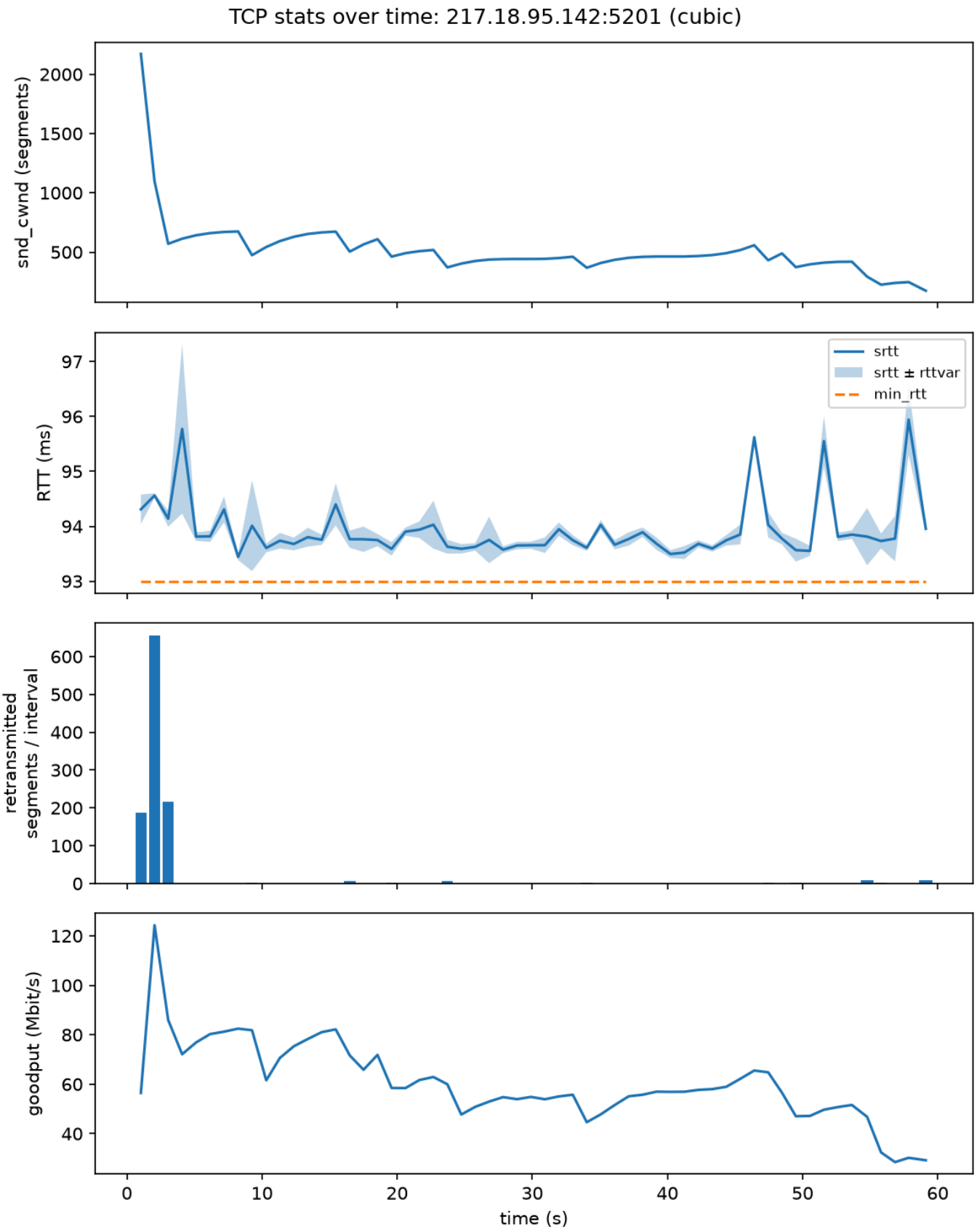

TCP stats over time: 217.18.95.142:5201 (cubic)
snd_cwnd (segments)
2000
1500
1000
500
RTT (ms)
97
96
95
94
93
srtt
srtt ± rttvar
min_rtt
retransmitted segments / interval
600
500
400
300
200
100
0
goodput (Mbit/s)
120
100
80
60
40
0
10
20
30
40
50
60
time (s)

Figure 1: snd_cwnd, RTT, retransmitted segments, and goodput over time for 217.18.95.142:5201 (CUBIC). Generated by `plot_tcp_stats()`.

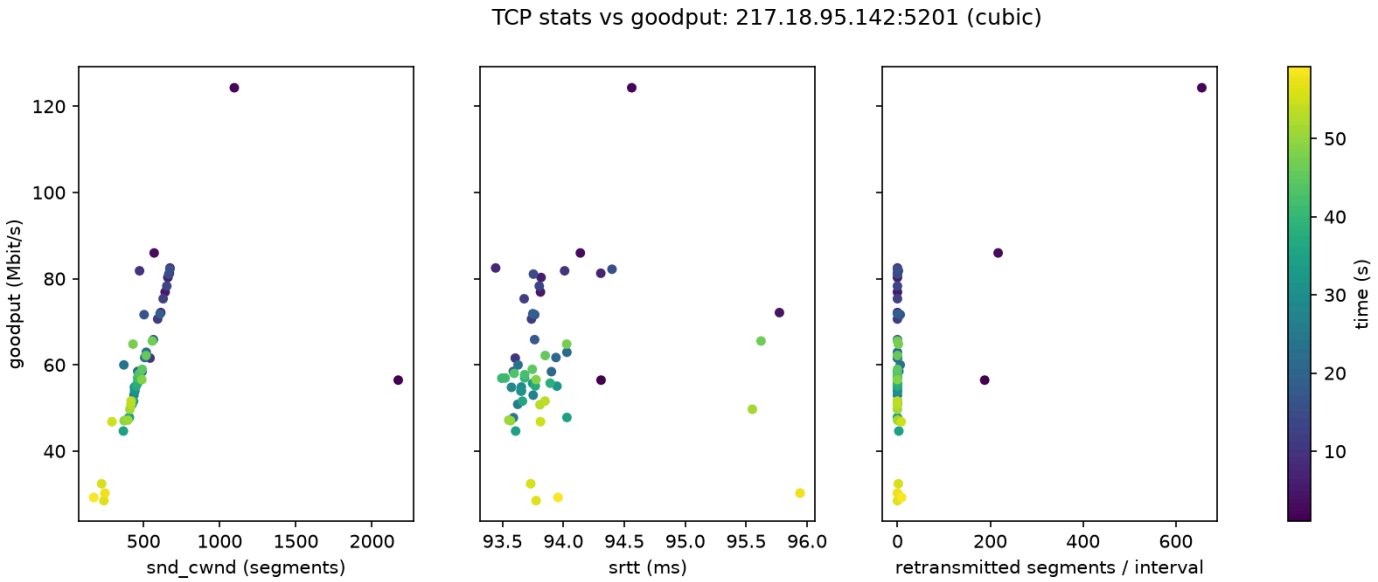


Figure 2: snd_cwnd, srtt, and retransmitted segments vs goodput for 217.18.95.142:5201 (CUBIC), colored by time. Generated by `plot_tcp_stats()`.

(c) Observations

Metric	Meaning	Expected effect on goodput in congestion avoidance
snd_cwnd	Max number of unacknowledged segments the sender can have in flight	Goodput ≈ cwnd × MSS / RTT until cwnd reaches the BDP. Past that, extra cwnd just fills the bottleneck queue.
srtt	Smoothed RTT estimate	For a fixed cwnd, higher RTT means lower goodput. srtt rising above min_rtt means packets are queueing.
Retransmitted segments	Our loss proxy	Each loss event makes CUBIC cut cwnd to 0.7×, so goodput should drop after retransmissions.
Goodput	Bytes acknowledged per second	The result the other metrics explain.

On this path goodput was mostly determined by cwnd. After slow start, goodput tracked cwnd closely (r = 0.94), and in Figure 2 the cwnd-vs-goodput panel is close to a straight line. For example, at 8.2 s, 674 segments × 1448 B × 8 / 93.4 ms = 83.6 Mbit/s, compared to a measured 82.5 Mbit/s. srtt stayed between 93.4 and 96.0 ms against a 93.0 ms minimum, so there was almost no queueing and srtt had no real relationship with goodput (r = −0.02).

The loss events behaved like CUBIC should. After each loss, ssthresh dropped to about 0.7× the previous cwnd (e.g. 674→471, 518→364, 419→293), and cwnd then grew back in the concave CUBIC shape, flattening out near the old maximum between 27 and 43 s. The retransmission count by itself didn't predict goodput well (r = −0.24), since most intervals had zero retransmissions and each loss event only retransmitted 2–9 segments, but each one still cost 30% of cwnd. There were no timeouts in any of the 10 traces (`rto_backoffs` was always 0).

Anomalies:

- **Slow-start overshoot.** cwnd hit 2172 segments at t = 1 s, and 96% of the run's retransmissions happened in the first 3 s before cwnd fell to 570.
- **Downward drift.** Goodput fell from about 82 to 29 Mbit/s over the minute while ssthresh dropped from 555 to 174 segments, with no increase in RTT. Losses kept arriving before cwnd got back to its previous max, so CUBIC kept lowering its target. Since RTT stayed flat, these losses were probably from a shallow buffer, a policer, or cross traffic rather than our own queue, but the trace can't tell us which.
- **RTT spikes while cwnd was falling.** srtt jumped to around 96 ms at 46, 51, and 58 s while our cwnd was shrinking, which suggests other traffic at the bottleneck.
- **Other paths.** On 185.246.208.67:5201, srtt reached 308.6 ms against a 122.0 ms minimum. On 156.146.40.65:5201 the mean goodput (17.7 Mbit/s) was almost 4× the median (4.6 Mbit/s), so a few fast seconds carried most of the data.

Part 3: Algorithm comparison and design

(a) Comparing CUBIC, Reno, and BBR

Setup. Session `20261024T151345` (`results/run_metadata.json`), kernel `6.6.87.2-microsoft-standard-WSL2`, run with:

```
.venv/bin/python src/main.py data/listed_iperf3_servers.csv --n 5 --reps 3 --time 60 --interval 1.0 --out result
```

- **Algorithm selection.** The algorithm is set with `setsockopt(TCP_CONGESTION)` before the data socket connects, then read back and recorded in every row (`set_congestion_control()`).
- **Repetitions and order.** Each destination ran every algorithm **3 times**, so there are 45 tests of 60 s each, sampled every 1 s. The order rotated each repetition: run 1 was CUBIC → Reno → BBR, run 2 was Reno → BBR → CUBIC, and run 3 was BBR → CUBIC → Reno (`main()`). All tests to one destination ran back to back, with a 5 s pause between them.
- **Destinations.** A destination counted only if all 9 of its tests finished (a failed test was retried up to 2 times). It took 11 destinations to get 5. The others timed out, refused connections, or reported busy; one was dropped after its BBR test in run 2 failed 3 times. There was no shortfall.

Goodput summary in Mbit/s, pooling all 3 runs (`results/summary.csv` ; per-run values are in `results/summary_runs.csv`):

Destination	Algorithm	Min	Median	Mean	p95
139.60.160.88:5201	cubic	68.5	138.9	176.2	577.8
139.60.160.88:5201	reno	30.4	114.5	122.7	197.8
139.60.160.88:5201	bbr	141.9	802.9	747.0	897.4
23.249.54.234:30002	cubic	22.0	77.6	98.5	235.8
23.249.54.234:30002	reno	12.3	58.9	71.1	215.1

Destination	Algorithm	Min	Median	Mean	p95
23.249.54.234:30002	bbr	36.3	375.0	351.1	441.1
5.180.211.133:5200	cubic	11.4	69.6	76.9	186.7
5.180.211.133:5200	reno	6.6	33.5	43.8	98.5
5.180.211.133:5200	bbr	9.8	192.5	173.8	240.5
80.67.167.93:9200	cubic	36.5	92.1	98.7	169.9
80.67.167.93:9200	reno	8.1	40.0	52.6	181.3
80.67.167.93:9200	bbr	29.6	201.4	187.0	219.6
212.47.219.73:5201	cubic	16.2	69.6	85.8	185.9
212.47.219.73:5201	reno	9.1	34.2	51.6	184.2
212.47.219.73:5201	bbr	2.2	168.0	150.3	188.6

Per-run mean goodput (Mbit/s, runs 1 / 2 / 3), with path and TCP statistics pooled over the runs. Queueing is the median of srtt − min_rtt; the last column is retransmitted segments as a share of data segments sent.

Destination (min RTT)	Algorithm	Goodput per run	Queueing (ms)	Median cwnd (segs)	Retrans.
139.60.160.88 (23 ms)	cubic	173 / 182 / 174	0.7	292	0.19%
	reno	116 / 106 / 146	0.7	242	0.16%
	bbr	770 / 772 / 700	1.1	4009	0.58%
23.249.54.234 (48 ms)	cubic	114 / 97 / 85	1.0	327	0.13%
	reno	58 / 82 / 73	0.8	248	0.09%
	bbr	376 / 314 / 364	1.1	4151	0.77%
5.180.211.133 (90 ms)	cubic	62 / 85 / 84	0.6	583	0.18%
	reno	77 / 34 / 19	0.9	311	0.28%
	bbr	170 / 191 / 160	1.1	4028	0.74%
80.67.167.93 (98 ms)	cubic	107 / 75 / 114	1.0	831	0.15%
	reno	54 / 37 / 67	1.1	328	0.14%
	bbr	197 / 166 / 199	1.2	4068	0.30%
212.47.219.73 (107 ms)	cubic	90 / 55 / 112	2.6	715	0.23%
	reno	39 / 45 / 71	1.4	356	0.09%
	bbr	166 / 128 / 155	1.2	3979	0.16%

Throughput.

- **BBR had the highest goodput in all 15 run-by-destination comparisons**, about 3× CUBIC overall (322 vs 107 Mbit/s mean). CUBIC beat Reno in 14 of 15.
- **The paths weren't congested by a queue.** For every algorithm, median srtt sat only 0.6–2.6 ms above min_rtt, so the buffers along these paths almost never filled. CUBIC and Reno still saw losses (0.1–0.3% of segments), and since RTT wasn't rising, those losses were probably random loss, policers, or shallow buffers, the same pattern as Part 2. Each loss cut cwnd (to 0.7× for CUBIC, 0.5× for Reno), so neither algorithm got close to the BDP. For example, at 80.67.167.93 BBR sustained about 220 Mbit/s at 98 ms, a BDP of about 2.7 MB or 1,860 segments. CUBIC's median cwnd was 831 segments (45% of that) and Reno's was 328 (18%), and their goodput (99 and 53 Mbit/s) matches cwnd × MSS / RTT.
- **BBR ignores isolated losses.** It sets its rate from the measured bottleneck bandwidth and min RTT, so random loss doesn't shrink its window. Its cwnd stayed around 4,000 segments, about 2× the BDP, which is BBR's cwnd gain. It retransmitted more on four of the five paths (up to 0.77% of segments, 2–6× CUBIC) but lost very little goodput doing so.
- **CUBIC's advantage over Reno grows with RTT.** The CUBIC/Reno goodput ratio was 1.4× at 23 ms and 48 ms, 1.8× at 90 ms, 1.9× at 98 ms, and 1.7× at 107 ms. Reno adds 1 segment per RTT, so recovering after a cut takes longer the longer the RTT. CUBIC's growth depends on elapsed time rather than RTT, so on long paths it gets back to its old window much faster.
- **BBR's results are capped by our send buffer.** WSL's `tcp_wmem` maximum is 4 MB, which allows about 2.7 MB of data in flight. BBR's in-flight data sat at that limit on every path while the receiver advertised 3–45 MB. So BBR's goodput is about 2.7 MB × 8 / RTT: 940 Mbit/s predicted at 23 ms vs 897 measured (p95), and 220 vs 220 at 98 ms. The real path capacity may be higher, which means BBR's advantage is probably understated. CUBIC and Reno never got near this limit.

RTT.

- **No algorithm added much delay.** With such shallow queues, the differences were small: median queueing was 0.9 ms for Reno, 0.9 ms for CUBIC, and 1.1 ms for BBR.
- **Reno and CUBIC kept RTT slightly lower on most paths.** At 139.60.160.88 BBR's queueing was 1.1 ms vs 0.7 ms. BBR keeps up to 2× the BDP in flight and probes for more bandwidth, so it adds a small standing queue.
- **The exception was 212.47.219.73,** where CUBIC built the most queue (2.6 ms). CUBIC fills the buffer until a loss happens, while BBR periodically drains its queue to re-measure min RTT.

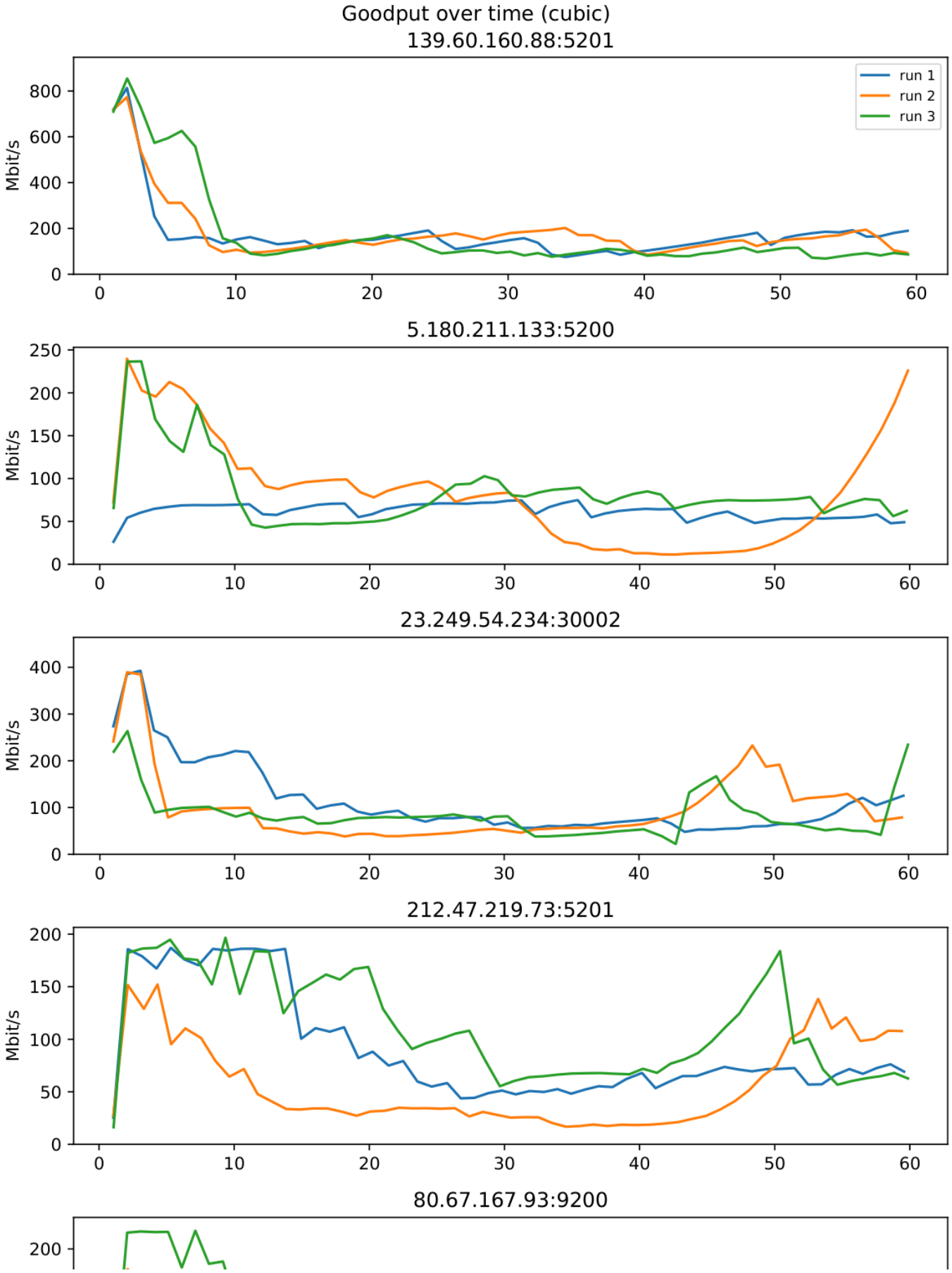
Operating regimes.

- **BBR does best** on paths with a large BDP, a shallow or unfilled buffer, and non-congestive loss. That describes every path we measured.
- **CUBIC is the better loss-based choice** on long-RTT paths, because its growth doesn't depend on RTT.
- **Reno is only competitive** on short-RTT paths, where it recovers quickly. Even there it was the slowest on every destination.
- **Where loss-based algorithms should look better:** at a deep, shared bottleneck, CUBIC and Reno would keep fewer retransmissions and be fairer to other flows. BBR's higher retransmission rate shows it ignoring loss signals that loss-based flows respond to. We couldn't test this, since none of our paths ever had a standing queue.

Variability and limits.

- **Run-to-run variation.** The coefficient of variation of a run's mean goodput across the 3 runs was 6–13% for BBR, 3–33% for CUBIC, and 17–69% for Reno. Reno at 5.180.211.133 fell from 77 to 34 to 19 Mbit/s over its three runs, even though it went second, first, and third in the order. That suggests the path itself changed over the ~30 minutes.
- **Test order mattered little.** Normalized to each destination and algorithm's mean, tests run first averaged 0.96×, second 1.03×, and third 1.00×.
- **Paths changed between tests.** At 212.47.219.73, min_rtt was 112 ms during the BBR runs but 107 ms during CUBIC and Reno, which points to a route change.
- **Limits of public-Internet comparisons.**
  - Tests ran one after another, not simultaneously, so each algorithm saw different cross traffic.
  - We don't control or know the bottleneck link, its buffer size, or whether a policer is in the path.
  - Three runs per destination and five destinations is a small sample.
  - Each flow ran alone, so this says nothing about how the algorithms behave when competing with each other.
  - Results depend on the host. Here the 4 MB send buffer limited BBR, and WSL2's virtual NIC adds its own small delay.

Plots. Goodput plots are generated by `plot_goodput()`; TCP stats plots by `plot_tcp_stats()` for the representative destination 80.67.167.93:9200, chosen by `pick_representative()` and shared by all three algorithms (`main()`). The source PDFs are `results/goodput_<algorithm>.pdf`, `results/tcp_stats_timeseries_<algorithm>.pdf`, and `results/tcp_stats_scatter_<algorithm>.pdf`.


Goodput over time (cubic)
139.60.160.88:5201
run 1
run 2
run 3
Mbit/s
0
200
400
600
800
0
10
20
30
40
50
60
5.180.211.133:5200
Mbit/s
0
50
100
150
200
250
0
10
20
30
40
50
60
23.249.54.234:30002
Mbit/s
0
100
200
300
400
0
10
20
30
40
50
60
212.47.219.73:5201
Mbit/s
0
50
100
150
200
0
10
20
30
40
50
60
80.67.167.93:9200
200

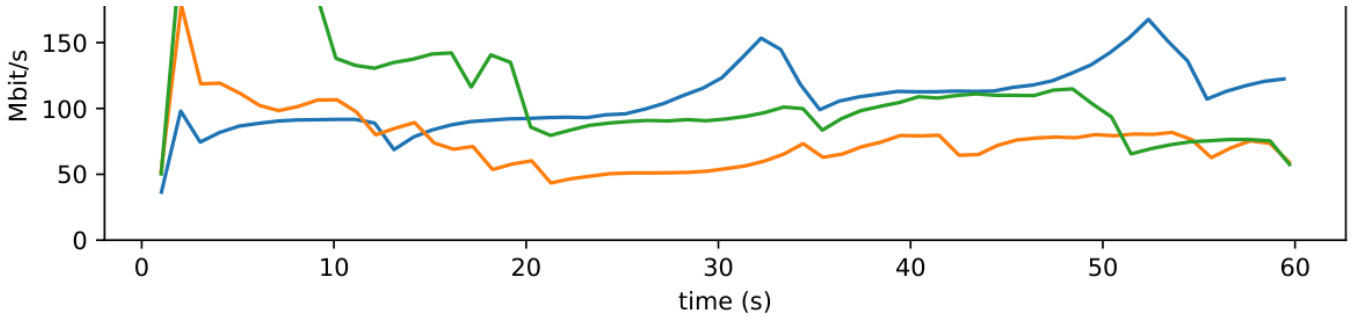
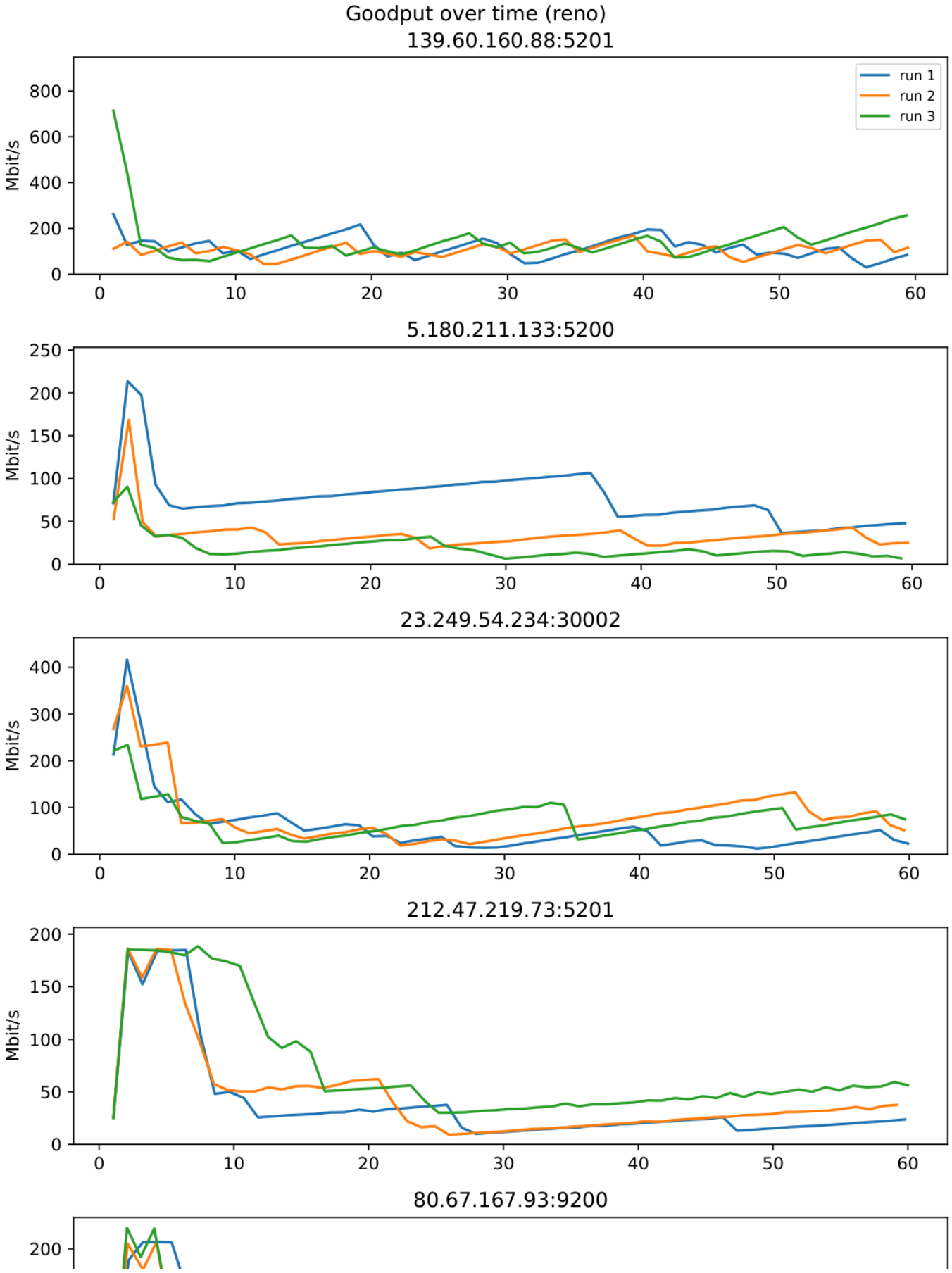


Figure 3a: Goodput over time per destination (CUBIC), one line per run. Each destination's y-axis is the same across 3a–3c.


Goodput over time (reno)
139.60.160.88:5201
run 1
run 2
run 3
Mbit/s
0
200
400
600
800
0
10
20
30
40
50
60
5.180.211.133:5200
Mbit/s
0
50
100
150
200
250
0
10
20
30
40
50
60
23.249.54.234:30002
Mbit/s
0
100
200
300
400
0
10
20
30
40
50
60
212.47.219.73:5201
Mbit/s
0
50
100
150
200
0
10
20
30
40
50
60
80.67.167.93:9200
200

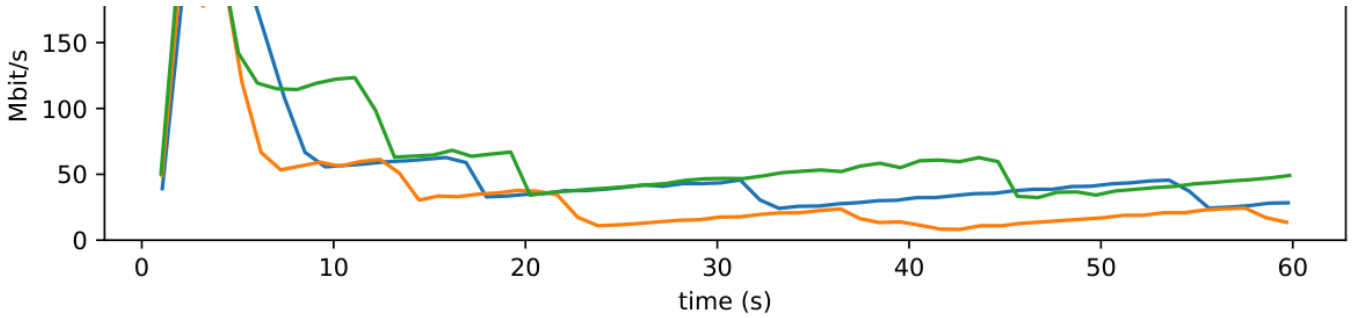
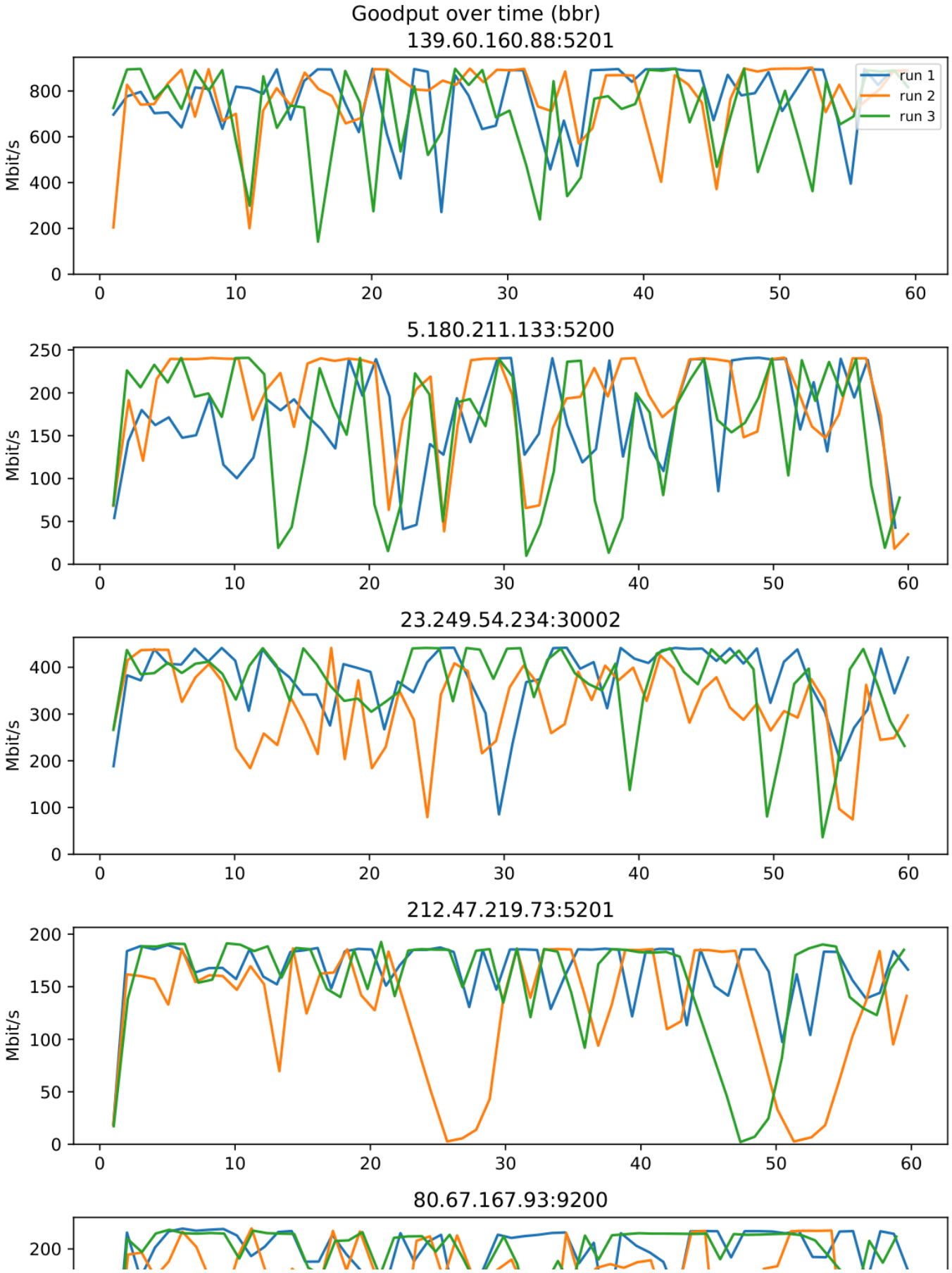


Figure 3b: Goodput over time per destination (Reno).


Goodput over time (bbr)
139.60.160.88:5201
Mbit/s
run 1
run 2
run 3
5.180.211.133:5200
23.249.54.234:30002
212.47.219.73:5201
80.67.167.93:9200

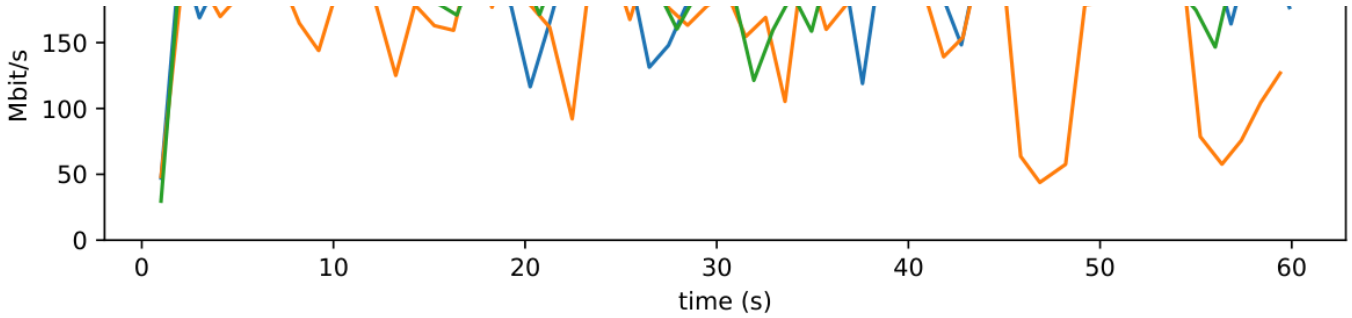


Figure 3c: Goodput over time per destination (BBR).

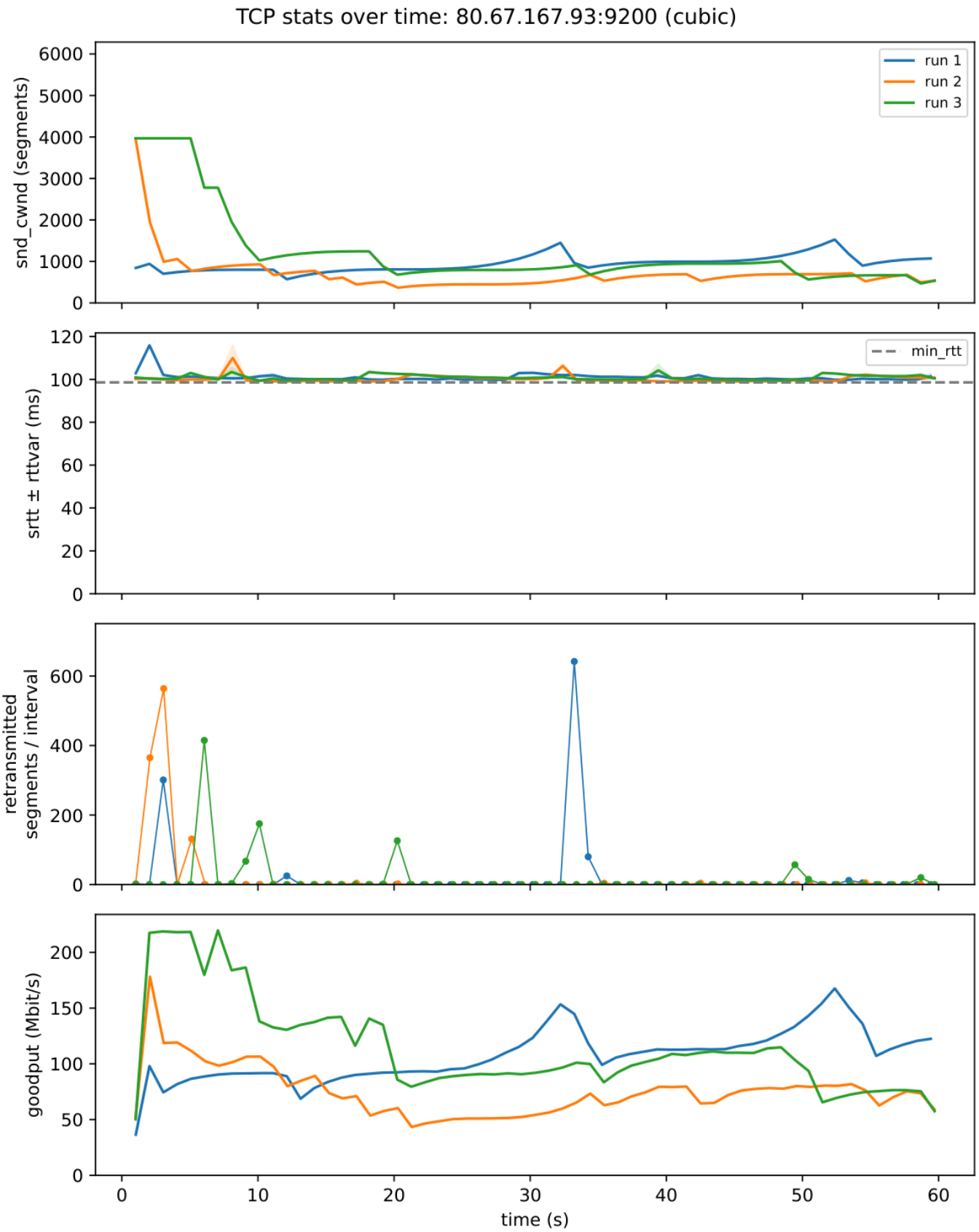

TCP stats over time: 80.67.167.93:9200 (cubic)
snd_cwnd (segments)
run 1
run 2
run 3
srtt ± rttvar (ms)
min_rtt
retransmitted segments / interval
goodput (Mbit/s)
time (s)

Figure 4a: snd_cwnd, RTT, retransmitted segments, and goodput over time, 80.67.167.93:9200 (CUBIC), one line per run. All three algorithms' plots use the same axis scales.

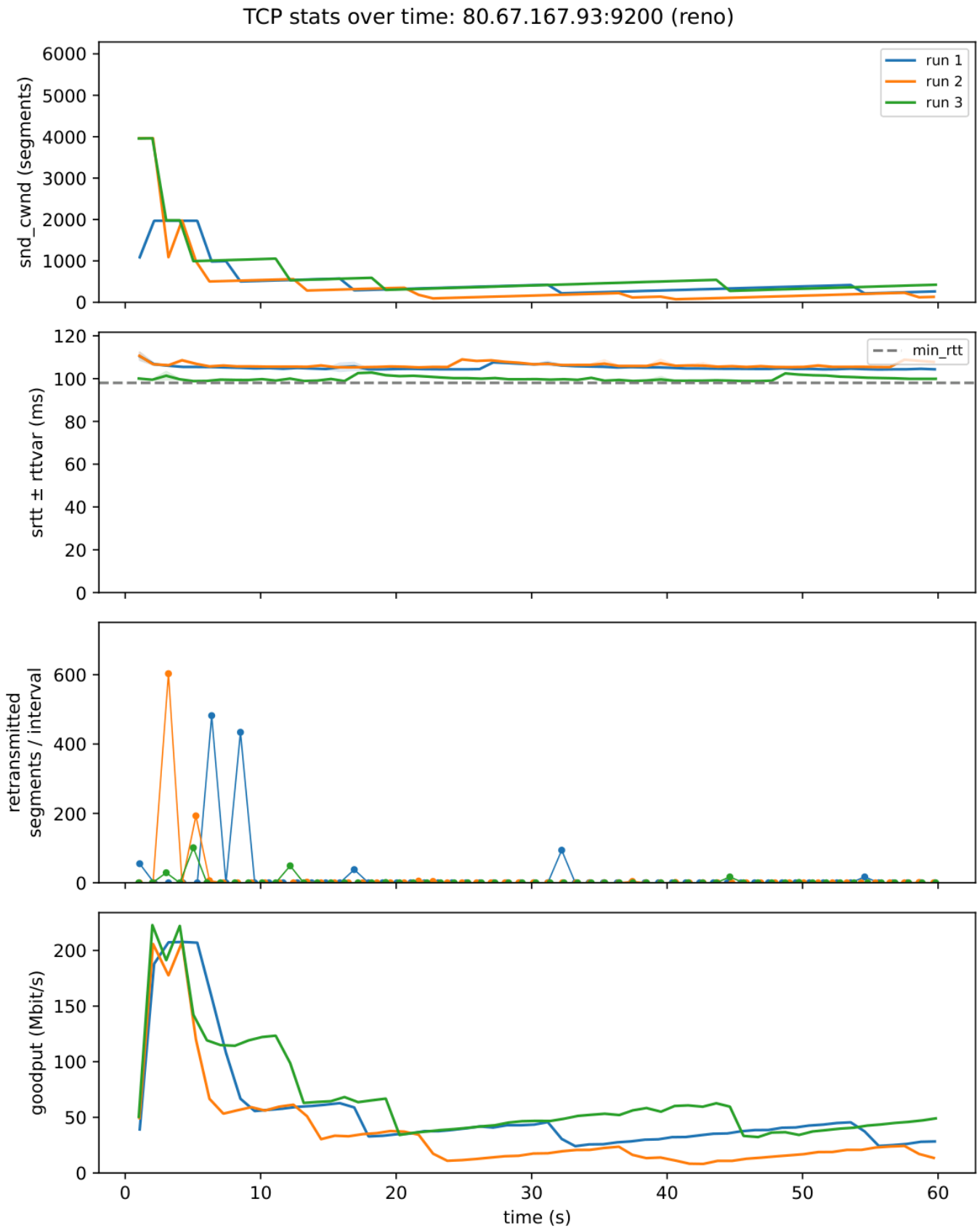


Figure 4b: Same as 4a (Reno).

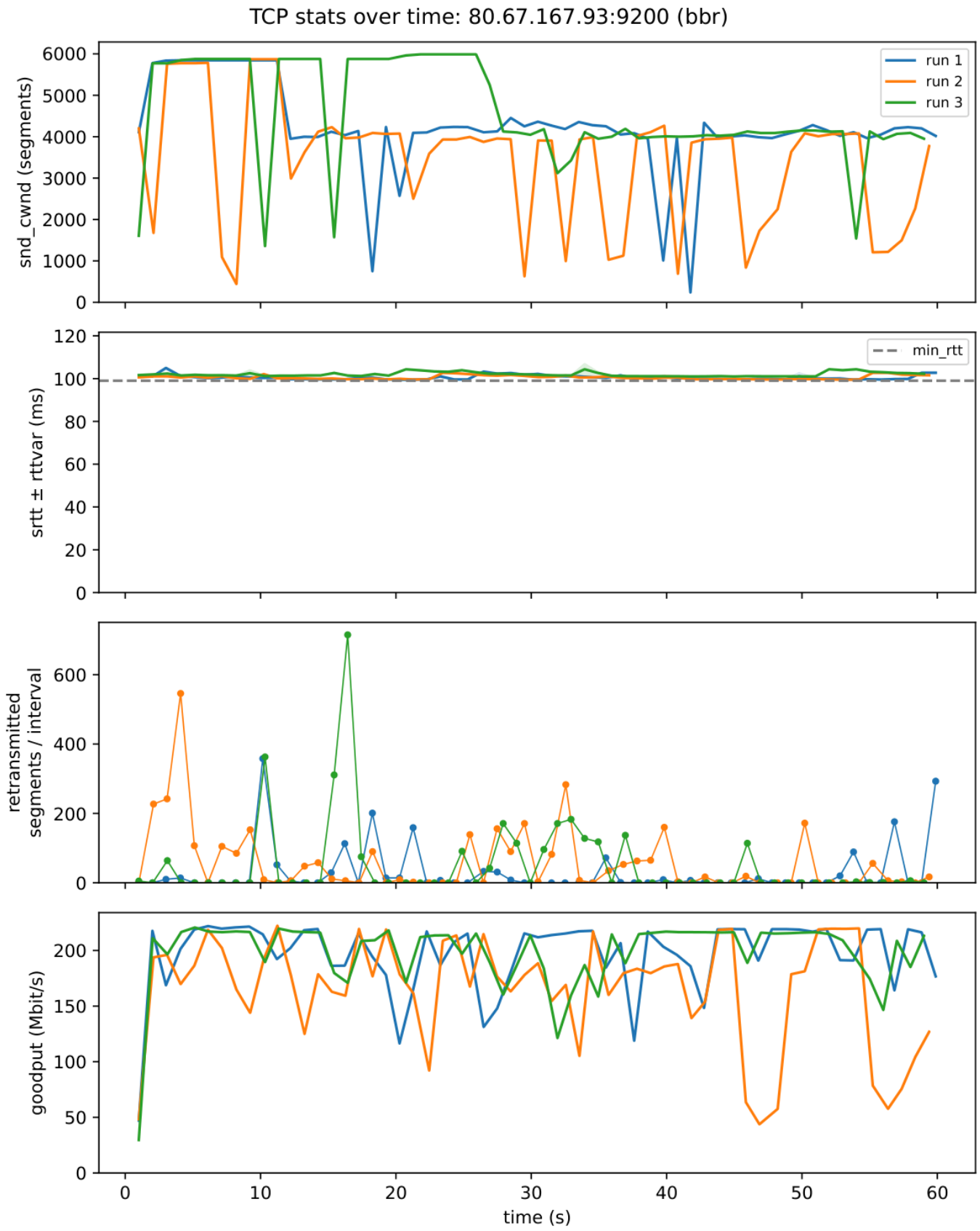


Figure 4c: Same as 4a (BBR).

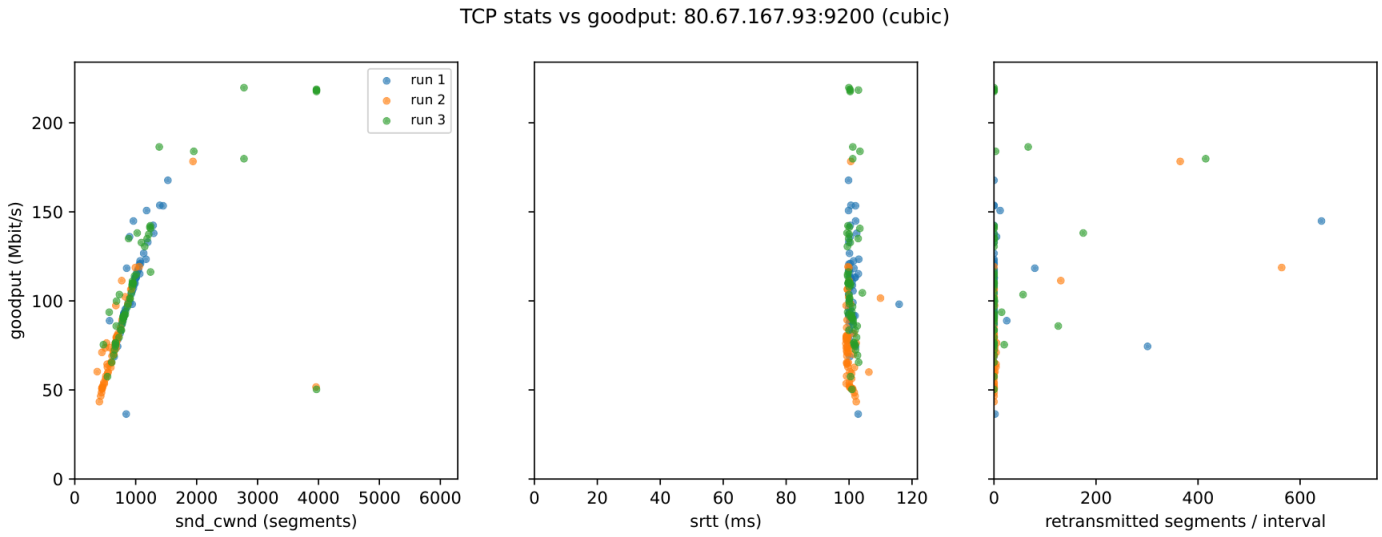


Figure 5a: snd_cwnd, srtt, and retransmitted segments vs goodput, 80.67.167.93:9200 (CUBIC), colored by run. All three algorithms' plots use the same axis scales.

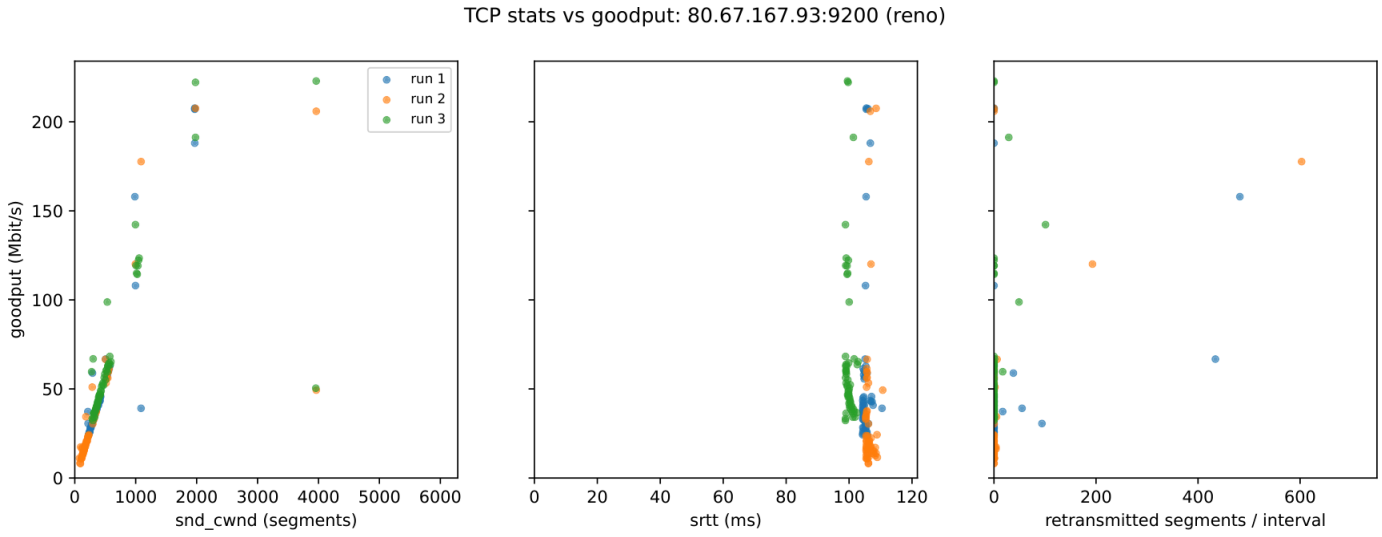


Figure 5b: Same as 5a (Reno).

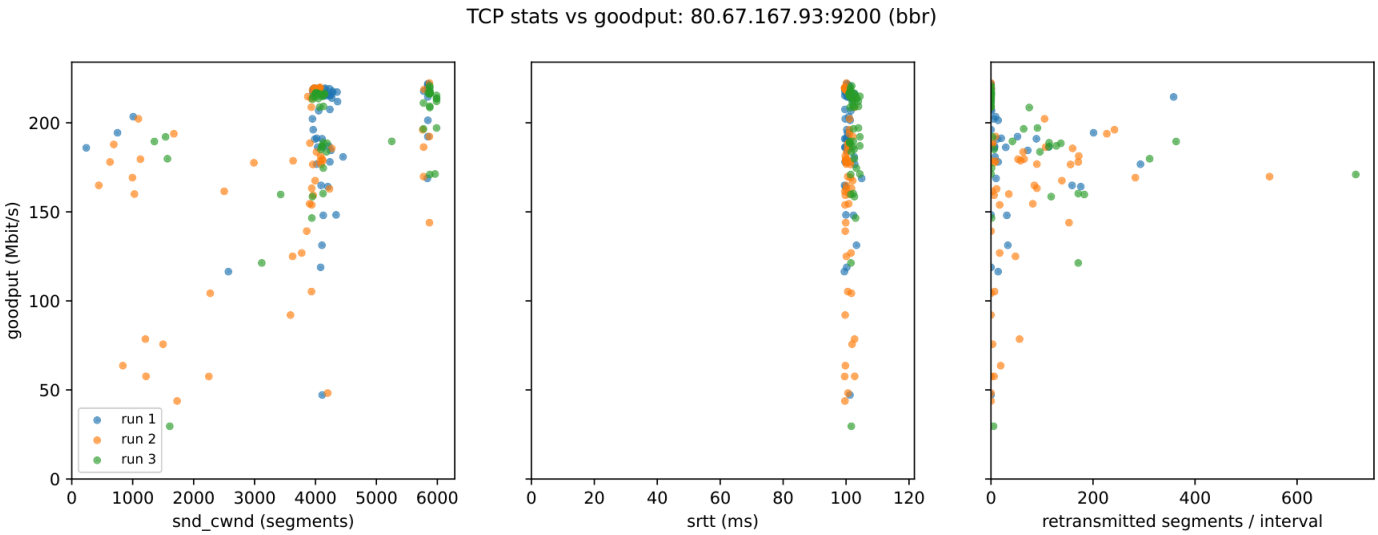


Figure 5c: Same as 5a (BBR).

(b) Congestion control design

```
Start up with the minimum window size

ONCE EVERY ROUND TRIP
  1. Measure:
     - base delay    = fastest round trip seen recently
     - current delay = smoothed round trip time
     - extra delay   = how far current delay is above base delay
     - goodput       = how much data got through this round trip

  2. If the connection timed out:
       Shrink window to minimum, go back to start-up mode.

  3. Else if packets were lost:
       If extra delay is large → shrink window to 60% (a real queue built up)
       If extra delay is small → shrink window to 90% (probably random loss)
       Leave start-up mode.

  4. Else if in start-up mode:
       If extra delay is getting noticeable → leave start-up mode
       Otherwise → double the window

  5. Else (normal operation):
       If extra delay is below target:
           Grow the window by a few packets.
           Grow more when far below target, less when close to it.
           If the last increase didn't raise goodput → grow only slightly.
       If extra delay is above target:
           Shrink the window by a percentage.
           Shrink more the further above target it is (never more than half).

  6. Never let the window be smaller than the minimum window size
     or bigger than about twice what the path can hold.
```

Parameters. The pseudocode leaves a few values open; these are the ones we would use.

Quantity	Value
Minimum window	4 segments
Base delay	Minimum RTT over the last 10 s
Target extra delay	max(5 ms, 10% of base delay)
"Large" extra delay on loss	Above the target
Leave start-up	Extra delay above half the target, or any loss
Growth per RTT	1 + 7·(target − extra)/target segments (1 to 8); 1 segment if the last increase didn't raise goodput
Shrink above target	cwnd × max(0.5, 1 − 0.5·(extra − target)/extra)
Upper cap	2 × (max recent goodput × base delay), i.e. twice the estimated BDP

Design choices.

- **Delay as the main signal.** In Part 2, goodput tracked cwnd almost linearly (r = 0.94) while srtt stayed within 3 ms of min_rtt. That means the path wasn't queueing, so growing cwnd directly increased goodput. Extra delay is the earliest sign that cwnd has passed the BDP (the bandwidth-delay product: how much data the path holds in flight) and is filling a queue. Reacting to it keeps the window near the BDP without first having to cause a loss.
- **Not every loss is congestion.** In Part 2, CUBIC cut cwnd to 0.7× on every loss even though RTT never rose, so ssthresh fell from 555 to 174 segments and goodput dropped from 82 to 29 Mbit/s. Those losses didn't come from our own queue. Part 3 showed the same pattern on all five paths: queueing stayed under 3 ms, yet loss-based cuts kept CUBIC and Reno at roughly 14–45% of the BDP, while BBR, which ignores those losses, got 1.7–4.2× CUBIC's goodput. Our design cuts by only 10% when a loss arrives without extra delay, and by 40% when the queue has actually grown.
- **Growth scales with headroom.** Reno adds one segment per RTT, so on long paths it takes a long time to reach the BDP. Growing by up to 8 segments when the queue is empty closes that gap faster, and the growth tapers off near the target so the window doesn't overshoot.
- **A goodput check.** If a larger window didn't increase goodput, the flow is already at the bottleneck rate, even if the buffer hasn't built up enough delay to show it yet. Growth then drops to 1 segment per RTT.
- **Start-up stops early.** In Part 2, slow start pushed cwnd to 2172 segments, and 96% of the run's retransmissions happened in the first 3 s. Leaving start-up as soon as extra delay appears avoids that burst of losses.
- **Bounded window.** The 2 × BDP cap stops the window from growing without limit when delay can't be measured, for example when ACKs are compressed or the path is application-limited.

Fairness and convergence. In normal operation the window grows additively and shrinks multiplicatively, which is the AIMD (additive-increase, multiplicative-decrease) structure. Each cut reduces a larger window by more segments than a smaller one, so the gap between two competing flows shrinks every round. Both flows see the same queueing delay at a shared bottleneck, so they also slow down at the same moments.

The stable operating point is where extra delay sits at the target. The queue there holds about target × bottleneck rate bytes, and each of N flows holds about 1/N of BDP + queue. Near the target, growth and shrinking both get small, so the window settles instead of moving in a sawtooth like Reno or CUBIC.

Two things weaken this:

- **RTT unfairness.** The window updates once per RTT, so a flow with a shorter RTT grows faster and takes a larger share. Reno and CUBIC have the same problem.
- **The latecomer problem.** A flow that starts while a queue already exists measures a base delay that's too high. It then tolerates more queueing and takes more bandwidth than its share. TCP Vegas and LEDBAT have this problem. Expiring the base delay after 10 s limits it but doesn't fix it.

Assumptions.

- The minimum RTT over 10 s is close to the true propagation delay.
- Queueing at the bottleneck is the main source of RTT variation.
- A loss without extra delay comes from a shallow buffer, a policer, or cross traffic rather than our own queue.
- The sender gets an RTT sample every round trip, as `tcp_info` provides.

Limitations.

- **Competing with loss-based flows.** Against CUBIC or Reno, which fill the buffer until a loss happens, the extra delay stays above our target. Our flow keeps shrinking and gets much less than its share. BBR behaves differently: it ignores most loss, which is how it got more goodput in our runs but also far more retransmissions.
- **Delay can come from somewhere other than our queue.** Route changes, Wi-Fi retransmissions, or a stretched ACK clock raise RTT without any congestion. The algorithm would wrongly slow down.
- **Loss classification can be wrong.** With a very shallow buffer, real congestion losses happen before any extra delay shows up. Those losses get only the mild 10% cut, so the flow could keep overloading the link.
- **Untested.** We only evaluated the design on paper against our traces; we never implemented or ran it.

AI use

Generated code and assisted with report + analysis using Codex and Claude Code.